

NeuralNetwork

Generated by Doxygen 1.15.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 ActivationFunction Struct Reference	5
3.1.1 Detailed Description	5
3.2 ColumnData Struct Reference	5
3.2.1 Detailed Description	6
3.3 Configuration Struct Reference	6
3.3.1 Detailed Description	6
3.4 DataPreprocessor Class Reference	6
3.4.1 Detailed Description	7
3.4.2 Member Function Documentation	7
3.4.2.1 fit()	7
3.4.2.2 interpret_extracted_column()	8
3.4.2.3 transform_and_extract()	8
3.5 LossFunction Struct Reference	9
3.5.1 Detailed Description	9
3.6 NeuralNetwork Class Reference	9
3.6.1 Detailed Description	10
3.6.2 Member Function Documentation	10
3.6.2.1 feedforward()	10
3.6.2.2 get_activations()	10
3.6.2.3 get_biases()	10
3.6.2.4 get_weights()	11
3.6.2.5 initialize_weights_and_biases()	11
3.6.2.6 load_model_from_file()	11
3.6.2.7 save_model_to_file()	12
3.6.2.8 test_model()	12
3.6.2.9 train()	13
4 File Documentation	15
4.1 NeuralNetworkProjectFoCP/data_preprocessor.h File Reference	15
4.1.1 Detailed Description	15
4.2 data_preprocessor.h	15
4.3 NeuralNetworkProjectFoCP/display.h File Reference	16
4.3.1 Detailed Description	16
4.3.2 Function Documentation	16
4.3.2.1 print_feedforward_output()	16
4.3.2.2 print_header()	17

4.3.2.3 print_vector()	17
4.4 display.h	17
4.5 NeuralNetworkProjectFoCP/file_data_operations.h File Reference	18
4.5.1 Detailed Description	18
4.5.2 Function Documentation	19
4.5.2.1 get_floats_from_line()	19
4.5.2.2 get_strings_from_line()	19
4.5.2.3 load_vector_from_file() [1/3]	19
4.5.2.4 load_vector_from_file() [2/3]	20
4.5.2.5 load_vector_from_file() [3/3]	20
4.5.2.6 parseCSV()	20
4.5.2.7 save_vector_to_file() [1/3]	21
4.5.2.8 save_vector_to_file() [2/3]	21
4.5.2.9 save_vector_to_file() [3/3]	21
4.6 file_data_operations.h	22
4.7 NeuralNetworkProjectFoCP/math_functions.h File Reference	22
4.7.1 Detailed Description	23
4.7.2 Function Documentation	23
4.7.2.1 cross_entropy()	23
4.7.2.2 cross_entropy_derivative()	24
4.7.2.3 mean_squared_error()	24
4.7.2.4 mean_squared_error_derivative()	25
4.7.2.5 mtanh()	25
4.7.2.6 relu()	25
4.7.2.7 relu_derivative()	26
4.7.2.8 sigmoid()	26
4.7.2.9 sigmoid_derivative()	26
4.7.3 Variable Documentation	27
4.7.3.1 activation_map	27
4.7.3.2 loss_function_map	27
4.8 math_functions.h	27
4.9 NeuralNetworkProjectFoCP/neural_network.h File Reference	28
4.9.1 Detailed Description	28
4.10 neural_network.h	28
4.11 resource.h	29
4.12 resource.h	29
4.13 pch.h	29

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ActivationFunction	Wrapper for an activation function and its corresponding derivative. [.	5
ColumnData	Metadata for an individual dataset column	5
Configuration	Holds all runtime settings for the neural network tool	6
DataPreprocessor	Handles the conversion of raw CSV string data into normalized float matrices	6
LossFunction	Wrapper for a loss function and its corresponding derivative	9
NeuralNetwork	Implementation of a multilayer perceptron with backpropagation capabilities	9

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

NeuralNetworkProjectFoCP/ data_preprocessor.h	
Logic for dataset cleaning, normalization, and one-hot encoding	15
NeuralNetworkProjectFoCP/ display.h	
Header file for display functions	16
NeuralNetworkProjectFoCP/ file_data_operations.h	
Declaration of functions for parsing, loading, and saving data to files	18
NeuralNetworkProjectFoCP/ math_functions.h	
Declaration of various mathematical functions and their derivatives	22
NeuralNetworkProjectFoCP/ neural_network.h	
Core Neural Network class definition and lifecycle management	28
NeuralNetworkProjectFoCP/ resource.h	29
UnitTest1/ pch.h	29
UnitTest1/ resource.h	29

Chapter 3

Class Documentation

3.1 ActivationFunction Struct Reference

Wrapper for an activation function and its corresponding derivative. [.

```
#include <math_functions.h>
```

Public Attributes

- `std::function< std::vector< float >(const std::vector< float > &>> func`
- `std::function< std::vector< float >(const std::vector< float > &>> derivative`

3.1.1 Detailed Description

Wrapper for an activation function and its corresponding derivative. [.

The documentation for this struct was generated from the following file:

- [NeuralNetworkProjectFoCP/math_functions.h](#)

3.2 ColumnData Struct Reference

Metadata for an individual dataset column.

```
#include <data_preprocessor.h>
```

Public Attributes

- `bool isNumeric`
- `std::vector< std::string > categories`
- `std::unordered_map< std::string, int > category_to_index`
- `float range [2]`

3.2.1 Detailed Description

Metadata for an individual dataset column.

The documentation for this struct was generated from the following file:

- [NeuralNetworkProjectFoCP/data_preprocessor.h](#)

3.3 Configuration Struct Reference

Holds all runtime settings for the neural network tool.

Public Attributes

- `std::string initialization_method` = "Xavier"
- `int hidden_layers` = 1
- `int neurons_nb` = 2
- `std::vector< std::string > activations` = {}
- `std::string train_data_path` = "iris.csv"
- `std::string target_column` = "species"
- `float test_fraction` = 0.2f
- `int epochs` = 140
- `float learning_rate` = 0.3f
- `std::string model_save_path` = ""
- `std::string model_load_path` = ""

3.3.1 Detailed Description

Holds all runtime settings for the neural network tool.

This structure is populated by the command-line parser and determines the architecture, data paths, and training hyperparameters.

The documentation for this struct was generated from the following file:

- [NeuralNetworkProjectFoCP/main.cpp](#)

3.4 DataPreprocessor Class Reference

Handles the conversion of raw CSV string data into normalized float matrices.

```
#include <data_preprocessor.h>
```

Public Member Functions

- void **clear** ()
Clears all the member variables of the [DataPreprocessor](#).
- void **fit** (const std::vector< std::vector< std::string > > &data)
Fits the preprocessor by determining column types and calculating normalization ranges.
- std::pair< std::vector< std::vector< float > >, std::vector< std::vector< float > > > **transform_and_extract** (const std::vector< std::vector< std::string > > &data, const std::string &column_to_extract, bool is_there_header=true)
Transforms the dataset using One-Hot Encoding and Min-Max Normalization.
- void **interpret_extracted_column** (const std::vector< float > &transformed_data) const
Decodes processed float data back into human-readable format.
- void **print_state** () const
Prints the current state of the [DataPreprocessor](#).

3.4.1 Detailed Description

Handles the conversion of raw CSV string data into normalized float matrices.

3.4.2 Member Function Documentation

3.4.2.1 fit()

```
void DataPreprocessor::fit (
    const std::vector< std::vector< std::string > > & data)
```

Fits the preprocessor by determining column types and calculating normalization ranges.

Iterates through the dataset to identify numeric vs. categorical columns. For numeric columns, it finds min/max for normalization. For categorical, it maps unique strings to indices.

Parameters

<i>data</i>	Dataset (2D string vector). The first row must be the header.
-------------	--

Note

This function populates internal maps and clears any previous state.

3.4.2.2 interpret_extracted_column()

```
void DataPreprocessor::interpret_extracted_column (
    const std::vector< float > & transformed_data) const
```

Decodes processed float data back into human-readable format.

Reverses normalization for numeric columns and picks the highest probability category (Argmax) for one-hot encoded categorical columns.

Parameters

<i>transformed_data</i>	A 1D vector representing a single sample of the extracted column.
-------------------------	---

Precondition

[transform_and_extract\(\)](#) must have been called at least once to set the active column name.

3.4.2.3 transform_and_extract()

```
std::pair< std::vector< std::vector< float > >, std::vector< std::vector< float > > >
DataPreprocessor::transform_and_extract (
    const std::vector< std::vector< std::string > > & data,
    const std::string & column_to_extract,
    bool is_there_header = true)
```

Transforms the dataset using One-Hot Encoding and Min-Max Normalization.

Separates the dataset into a feature set and a specific target column.

Parameters

<i>data</i>	The raw string data to transform.
<i>column_to_extract</i>	The name of the column to be used as the target (Y).
<i>is_there_header</i>	Flag indicating if the input 'data' includes a header row.

Returns

A pair of 2D float vectors:

- **first**: Transformed feature matrix (all columns except the extracted one).
- **second**: Transformed target matrix (the extracted column).

Precondition

The preprocessor must have been successfully fitted using [fit\(\)](#).

The documentation for this class was generated from the following files:

- [NeuralNetworkProjectFoCP/data_preprocessor.h](#)
- [NeuralNetworkProjectFoCP/data_preprocessor.cpp](#)

3.5 LossFunction Struct Reference

Wrapper for a loss function and its corresponding derivative.

```
#include <math_functions.h>
```

Public Attributes

- `std::function< std::vector< float >(const std::vector< float > &, const std::vector< float > &)>` **func**
- `std::function< std::vector< float >(const std::vector< float > &, const std::vector< float > &)>` **derivative**

3.5.1 Detailed Description

Wrapper for a loss function and its corresponding derivative.

The documentation for this struct was generated from the following file:

- [NeuralNetworkProjectFoCP/math_functions.h](#)

3.6 NeuralNetwork Class Reference

Implementation of a multilayer perceptron with backpropagation capabilities.

```
#include <neural_network.h>
```

Public Member Functions

- void [initialize_weights_and_biases](#) (int input_size, int hidden_layers_number, int neurons_per_hidden_layer, int output_size, std::string initialization_method="Xavier", std::vector< std::string > activation_functions={}, int seed=std::random_device())
Initialize weights and biases for the neural network.
- void **visualize_model** () const
Visualize the neural network model structure (weights and biases).
- std::vector< std::vector< float > > [feedforward](#) (const std::vector< std::vector< float > > &input) const
Performs a forward pass through the network for multiple input samples.
- void [train](#) (const std::vector< std::vector< float > > &inputs, const std::vector< std::vector< float > > &targets, const std::vector< std::vector< float > > &test_inputs, const std::vector< std::vector< float > > &test_targets, int epochs, float learning_rate, const std::string &loss_function_name)
Train the neural network using the provided training data.
- float [test_model](#) (const std::vector< std::vector< float > > &test_inputs, const std::vector< std::vector< float > > &test_targets, const std::string &loss_function_name) const
Calculates the network loss on a test dataset.
- void [save_model_to_file](#) (const std::string &filename) const
Saves the model state (weights, biases, activations) to 3 files: {filename}_weights.txt, {filename}_biases.txt, {filename}_activations.txt.
- void [load_model_from_file](#) (const std::string &filename)
Loads the model state (weights, biases, activations) from disk.
- const std::vector< std::vector< std::vector< float > > > & [get_weights](#) () const
Get the weights of the neural network.
- const std::vector< std::vector< float > > & [get_biases](#) () const
Get the biases of the neural network.
- const std::vector< std::string > & [get_activations](#) () const
Get the activation functions of the neural network.

3.6.1 Detailed Description

Implementation of a multilayer perceptron with backpropagation capabilities.

3.6.2 Member Function Documentation

3.6.2.1 feedforward()

```
std::vector< std::vector< float > > NeuralNetwork::feedforward (
    const std::vector< std::vector< float > > & input) const
```

Performs a forward pass through the network for multiple input samples.

Parameters

<i>input</i>	A 2D vector where each row is an input sample.
--------------	--

Returns

A 2D vector of predictions. Returns an empty 2D vector if the model is invalid.

Note

Ensure the input sample size matches the network's input layer size.

3.6.2.2 get_activations()

```
const std::vector< std::string > & NeuralNetwork::get_activations () const
```

Get the activation functions of the neural network.

Returns

A constant reference to the activation functions.

3.6.2.3 get_biases()

```
const std::vector< std::vector< float > > & NeuralNetwork::get_biases () const
```

Get the biases of the neural network.

Returns

A constant reference to the biases.

3.6.2.4 get_weights()

```
const std::vector< std::vector< std::vector< float > > > & NeuralNetwork::get_weights ()
const
```

Get the weights of the neural network.

Returns

A constant reference to the weights.

3.6.2.5 initialize_weights_and_biases()

```
void NeuralNetwork::initialize_weights_and_biases (
    int input_size,
    int hidden_layers_number,
    int neurons_per_hidden_layer,
    int output_size,
    std::string initialization_method = "Xavier",
    std::vector< std::string > activation_functions_passed = {},
    int seed = std::random_device()())
```

Initialize weights and biases for the neural network.

Supports Xavier and He initialization methods. Biases are initialized to zero.

Parameters

<i>input_size</i>	The size of the input layer.
<i>hidden_layers_number</i>	The number of hidden layers.
<i>neurons_per_hidden_layer</i>	The number of neurons in each hidden layer.
<i>output_size</i>	The size of the output layer.
<i>initialization_method</i>	The method used ("Xavier" or "He"). Defaults to Xavier if unknown.
<i>activation_functions_passed</i>	Activation functions for each layer (count should be <code>hidden_layers_number + 1</code>).
<i>seed</i>	The random seed for reproducibility.

3.6.2.6 load_model_from_file()

```
void NeuralNetwork::load_model_from_file (
    const std::string & filename)
```

Loads the model state (weights, biases, activations) from disk.

Parameters

<i>filename</i>	The base name of the files (appends <code>_weights.txt</code> , etc. automatically).
-----------------	--

Note

This function triggers `validate_model_structure()` after loading.

3.6.2.7 save_model_to_file()

```
void NeuralNetwork::save_model_to_file (
    const std::string & filename) const
```

Saves the model state (weights, biases, activations) to 3 files: {filename}_weights.txt, {filename}_biases.txt, {filename}_activations.txt.

Parameters

<i>filename</i>	The base name of the files (appends _weights.txt, etc. automatically).
-----------------	--

3.6.2.8 test_model()

```
float NeuralNetwork::test_model (
    const std::vector< std::vector< float > > & test_inputs,
    const std::vector< std::vector< float > > & test_targets,
    const std::string & loss_function_name) const
```

Calculates the network loss on a test dataset.

Parameters

<i>test_inputs</i>	Input samples for testing.
<i>test_targets</i>	Expected output labels.
<i>loss_function_name</i>	The name of the loss function (e.g., "MSE").

Returns

The average loss across all samples. Returns -1.0f if the model is invalid or sizes mismatch.

Parameters

<i>test_inputs</i>	Input samples for testing.
<i>test_targets</i>	Expected output labels.
<i>loss_function_name</i>	The name of the loss function ("MSE" or "BCE").

Returns

The average loss across all samples. Returns -1.0f if the model is invalid or sizes mismatch.

3.6.2.9 train()

```
void NeuralNetwork::train (
    const std::vector< std::vector< float > > & inputs,
    const std::vector< std::vector< float > > & targets,
    const std::vector< std::vector< float > > & test_inputs,
    const std::vector< std::vector< float > > & test_targets,
    int epochs,
    float learning_rate,
    const std::string & loss_function_name)
```

Train the neural network using the provided training data.

Parameters

<i>inputs</i>	The input data for training - each row represents a training sample. It is expected to be shuffled.
<i>targets</i>	The target output data for training - each row represents the target output for the corresponding training sample.
<i>test_inputs</i>	The input data for testing during training - each row represents a different test sample.
<i>test_targets</i>	The target output data for testing during training - each row represents the target output for the corresponding test sample.
<i>epochs</i>	The number of epochs to train for.
<i>learning_rate</i>	The learning rate for weight updates.
<i>loss_function_name</i>	The name of the loss function to use - right now only "MSE".

The documentation for this class was generated from the following files:

- NeuralNetworkProjectFoCP/[neural_network.h](#)
- NeuralNetworkProjectFoCP/[neural_network.cpp](#)

Chapter 4

File Documentation

4.1 NeuralNetworkProjectFoCP/data_preprocessor.h File Reference

Logic for dataset cleaning, normalization, and one-hot encoding.

```
#include <vector>
#include <string>
#include <unordered_map>
```

Classes

- struct [ColumnData](#)
Metadata for an individual dataset column.
- class [DataPreprocessor](#)
Handles the conversion of raw CSV string data into normalized float matrices.

4.1.1 Detailed Description

Logic for dataset cleaning, normalization, and one-hot encoding.

4.2 data_preprocessor.h

[Go to the documentation of this file.](#)

```
00001 #ifndef DATAPREPROCESSOR_H
00002 #define DATAPREPROCESSOR_H
00006 #include <vector>
00007 #include <string>
00008 #include <unordered_map>
00009
00014 struct ColumnData {
00015     bool isNumeric;
00016     std::vector<std::string> categories; // All unique categories for categorical columns
00017     std::unordered_map<std::string, int> category_to_index;
00018     float range[2]; // Min and Max for numeric columns
00019 };
00024 class DataPreprocessor {
00025 private:
00026     std::vector<ColumnData> columns; // Column index -> column_data
```

```

00027     std::unordered_map<std::string, int> column_order;
00028     std::string extracted_column_name;
00029     bool is_fitted = false;
00030
00031 public:
00032     void clear();
00043     void fit(const std::vector<std::vector<std::string>& data);
00055     std::pair<std::vector<std::vector<float>>, std::vector<std::vector<float>>>
    transform_and_extract(const std::vector<std::vector<std::string>& data, const std::string&
    column_to_extract, bool is_there_header = true);
00063     void interpret_extracted_column(const std::vector<float>& transformed_data) const;
00067     void print_state() const;
00068 };
00069
00070 #endif // DATAPREPROCESSOR_H

```

4.3 NeuralNetworkProjectFoCP/display.h File Reference

Header file for display functions.

```
#include <string>
```

Functions

- void `print_header` (const std::string &text)
Prints a stylized header to the console.
- void `print_vector` (const std::vector< float > &vec)
Prints a 1D vector of floats in a formatted horizontal array.
- void `print_feedforward_output` (const std::vector< std::vector< float > > &feedforward_input, const std::vector< std::vector< float > > &target_output, const std::vector< std::vector< float > > &feedforward_output, const size_t sample_count=5)
Displays a comparison between inputs, targets, and actual model predictions.

4.3.1 Detailed Description

Header file for display functions.

4.3.2 Function Documentation

4.3.2.1 `print_feedforward_output()`

```

void print_feedforward_output (
    const std::vector< std::vector< float > > & feedforward_input,
    const std::vector< std::vector< float > > & target_output,
    const std::vector< std::vector< float > > & feedforward_output,
    size_t sample_count = 5)

```

Displays a comparison between inputs, targets, and actual model predictions.

Useful for visually inspecting model performance on a small sample of data.

Parameters

<i>feedforward_input</i>	The original feature data.
<i>target_output</i>	The ground-truth labels.
<i>feedforward_output</i>	The predictions generated by the NeuralNetwork .
<i>sample_count</i>	The maximum number of samples to print (defaults to 5).

4.3.2.2 print_header()

```
void print_header (
    const std::string & text)
```

Prints a stylized header to the console.

Wraps the provided text in rows of '=' characters for visual separation.

Parameters

<i>text</i>	The string to be displayed within the header.
-------------	---

4.3.2.3 print_vector()

```
void print_vector (
    const std::vector< float > & vec)
```

Prints a 1D vector of floats in a formatted horizontal array.

Output format is [val1 val2 ...] with each value fixed to 2 decimal places and a width of 6 characters for alignment.

Parameters

<i>vec</i>	The vector of floats to display.
------------	----------------------------------

4.4 display.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #ifndef DISPLAY_H
00003 #define DISPLAY_H
00004
00005 #include <string>
00009
00010 // Prints a main header
00016 void print_header(const std::string& text);
00023 void print_vector(const std::vector<float>& vec);
00024
00033 void print_feedforward_output(const std::vector<std::vector<float>& feedforward_input,
00034                               const std::vector<std::vector<float>& target_output,
00035                               const std::vector<std::vector<float>& feedforward_output,
00036                               const size_t sample_count = 5);
00037 #endif
```

4.5 NeuralNetworkProjectFoCP/file_data_operations.h File Reference

Declaration of functions for parsing, loading, and saving data to files.

```
#include <vector>
#include <string>
#include <fstream>
```

Functions

- `std::vector< std::string > get\_strings\_from\_line (const std::string &line)`
Convert a line of comma-separated values into a vector of strings.
- `std::vector< float > get\_floats\_from\_line (const std::string &line)`
Get a vector of floats from a comma-separated line.
- `std::vector< std::vector< std::string > > parseCSV (const std::string &filename)`
Loading CSV file and parsing data into a string matrix.
- `void save\_vector\_to\_file (const std::string &filename, const std::vector< float > &vec, char delimiter=',')`
Save a 1D vector of floats to a file.
- `void save\_vector\_to\_file (const std::string &filename, const std::vector< std::string > &vec, char delimiter=',')`
Save a 1D vector of strings to a file. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.
- `void save\_vector\_to\_file (const std::string &filename, const std::vector< std::vector< float > > &vec, char delimiter=',')`
Save a 2D vector (matrix) of floats to a file.
- `void save\_vector\_to\_file (const std::string &filename, const std::vector< std::vector< std::vector< float > > > &vec, char delimiter=',')`
Save a 3D vector (list of matrices) of floats to a file.
- `void load\_vector\_from\_file (const std::string &filename, std::vector< std::string > &vec, char delimiter=',')`
Load a 1D vector of strings from a single-line file.
- `void load\_vector\_from\_file (const std::string &filename, std::vector< std::vector< float > > &vec, char delimiter=',')`
Load a 2D vector (matrix) of floats from a multi-line file.
- `void load\_vector\_from\_file (const std::string &filename, std::vector< std::vector< std::vector< float > > > &vec, char delimiter=',')`
Load a 3D vector (list of matrices) from a file.

4.5.1 Detailed Description

Declaration of functions for parsing, loading, and saving data to files.

4.5.2 Function Documentation

4.5.2.1 get_floats_from_line()

```
std::vector< float > get_floats_from_line (
    const std::string & line)
```

Get a vector of floats from a comma-separated line.

Parameters

<i>line</i>	String line containing comma-separated float values.
-------------	--

Returns

One-dimensional vector of floats.

4.5.2.2 get_strings_from_line()

```
std::vector< std::string > get_strings_from_line (
    const std::string & line)
```

Convert a line of comma-separated values into a vector of strings.

Parameters

<i>line</i>	String line containing comma-separated values.
-------------	--

Returns

One-dimensional vector of strings.

4.5.2.3 load_vector_from_file() [1/3]

```
void load_vector_from_file (
    const std::string & filename,
    std::vector< std::string > & vec,
    char delimiter)
```

Load a 1D vector of strings from a single-line file.

Parameters

	<i>filename</i>	The name of the file to read.
out	<i>vec</i>	The string vector to be populated (cleared before loading).
	<i>delimiter</i>	The character used to split the values.

4.5.2.4 load_vector_from_file() [2/3]

```
void load_vector_from_file (
    const std::string & filename,
    std::vector< std::vector< float > > & vec,
    char delimiter)
```

Load a 2D vector (matrix) of floats from a multi-line file.

Each non-empty line in the file is treated as a row in the matrix.

Parameters

out	vec	The 2D vector to be populated. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.
-----	-----	---

4.5.2.5 load_vector_from_file() [3/3]

```
void load_vector_from_file (
    const std::string & filename,
    std::vector< std::vector< std::vector< float > > > & vec,
    char delimiter)
```

Load a 3D vector (list of matrices) from a file.

Reconstructs the 3D structure by using empty lines as separators between different matrices.

Parameters

out	vec	The 3D vector to be populated. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.
-----	-----	---

4.5.2.6 parseCSV()

```
std::vector< std::vector< std::string > > parseCSV (
    const std::string & filename)
```

Loading CSV file and parsing data into a string matrix.

Note: This implementation currently reads all lines, including the header.

Parameters

filename	The name of the CSV file to parse.
----------	------------------------------------

Returns

A 2D vector (rows and columns) of strings.

4.5.2.7 save_vector_to_file() [1/3]

```
void save_vector_to_file (
    const std::string & filename,
    const std::vector< float > & vec,
    char delimiter)
```

Save a 1D vector of floats to a file.

Parameters

<i>filename</i>	Name of the file to create or overwrite.
<i>vec</i>	The 1D vector containing float data.
<i>delimiter</i>	The character used to separate values (default is comma).

4.5.2.8 save_vector_to_file() [2/3]

```
void save_vector_to_file (
    const std::string & filename,
    const std::vector< std::vector< float > > & vec,
    char delimiter)
```

Save a 2D vector (matrix) of floats to a file.

Each inner vector is written as a single line, with elements separated by the delimiter.

Parameters

<i>vec</i>	The 2D vector (matrix) of floats. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.
------------	--

4.5.2.9 save_vector_to_file() [3/3]

```
void save_vector_to_file (
    const std::string & filename,
    const std::vector< std::vector< std::vector< float > > > & vec,
    char delimiter)
```

Save a 3D vector (list of matrices) of floats to a file.

Matrices are separated by a blank line. Within each matrix, rows are separated by newlines and elements by the delimiter.

Parameters

<i>vec</i>	The 3D vector representing multiple layers/matrices. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.
------------	---

4.6 file_data_operations.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #ifndef FILE_DATA_OPERATIONS_H
00003 #define FILE_DATA_OPERATIONS_H
00007 #include <vector>
00008 #include <string>
00009 #include <fstream>
00010
00011 //Parsing functions
00017 std::vector<std::string> get_strings_from_line(const std::string& line);
00023 std::vector<float> get_floats_from_line(const std::string& line);
00024
00025 //Loading/saving to file functions
00032 std::vector<std::vector<std::string>> parseCSV(const std::string& filename);
00033
00034 // Overload for 1D vectors
00041 void save_vector_to_file(const std::string& filename,
00042     const std::vector<float>& vec,
00043     char delimiter = ',');
00048 void save_vector_to_file(const std::string& filename, const std::vector<std::string>& vec, char
    delimiter = ',');
00049 // Overload for 2D vectors
00056 void save_vector_to_file(const std::string& filename,
00057     const std::vector<std::vector<float>>& vec,
00058     char delimiter = ',');
00059 // Overload for 3D vectors
00067 void save_vector_to_file(const std::string& filename, const
    std::vector<std::vector<std::vector<float>>>& vec, char delimiter = ',');
00068
00069 // Overloads for loading from file
00076 void load_vector_from_file(const std::string& filename, std::vector<std::string>& vec, char delimiter
    = ',');
00077
00078 // 2D vector overload
00085 void load_vector_from_file(const std::string& filename, std::vector<std::vector<float>>& vec, char
    delimiter = ',');
00086 // 3D vector overload
00093 void load_vector_from_file(const std::string& filename, std::vector<std::vector<std::vector<float>>>&
    vec, char delimiter = ',');
00094
00095
00096 #endif

```

4.7 NeuralNetworkProjectFoCP/math_functions.h File Reference

Declaration of various mathematical functions and their derivatives.

```

#include <vector>
#include <string>
#include <map>
#include <functional>

```

Classes

- struct [ActivationFunction](#)
Wrapper for an activation function and its corresponding derivative. [
- struct [LossFunction](#)
Wrapper for a loss function and its corresponding derivative.

Functions

- `std::vector< float > relu` (`const std::vector< float > &x`)
Applies the Rectified Linear Unit (ReLU) activation: $f(x) = \max(0, x)$.
- `std::vector< float > relu_derivative` (`const std::vector< float > &x`)
Derivative of the ReLU function: $f'(x) = 1$ if $x > 0$ else 0 .
- `std::vector< float > sigmoid` (`const std::vector< float > &x`)
Applies the Sigmoid activation: $f(x) = \frac{1}{1+e^{-x}}$.
- `std::vector< float > sigmoid_derivative` (`const std::vector< float > &x`)
Derivative of the Sigmoid function: $f'(x) = f(x) \cdot (1 - f(x))$.
- `std::vector< float > mtanh` (`const std::vector< float > &x`)
Applies the Hyperbolic Tangent (Tanh) activation: $f(x) = \tanh(x)$.
- `std::vector< float > mtanh_derivative` (`const std::vector< float > &x`)
Applies the derivative of the Tanh activation on vector - $1 - \tanh^2(x)$.
- `std::vector< float > mean_squared_error` (`const std::vector< float > &predicted`, `const std::vector< float > &actual`)
Computes the element-wise Squared Error loss: $L = \frac{1}{2}(pred - actual)^2$.
- `std::vector< float > mean_squared_error_derivative` (`const std::vector< float > &predicted`, `const std::vector< float > &actual`)
Computes the derivative of the MSE loss with respect to the prediction.
- `std::vector< float > cross_entropy` (`const std::vector< float > &predicted`, `const std::vector< float > &actual`)
Binary Cross Entropy Loss: $L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$.
- `std::vector< float > cross_entropy_derivative` (`const std::vector< float > &predicted`, `const std::vector< float > &actual`)
Derivative of Binary Cross Entropy Loss with respect to predictions.

Variables

- `const std::map< std::string, ActivationFunction > activation_map`
Registry mapping string names (e.g., "relu") to [ActivationFunction](#) objects.
- `const std::map< std::string, LossFunction > loss_function_map`
Registry mapping string names (e.g., "MSE") to [LossFunction](#) objects.

4.7.1 Detailed Description

Declaration of various mathematical functions and their derivatives.

4.7.2 Function Documentation

4.7.2.1 cross_entropy()

```
std::vector< float > cross_entropy (
    const std::vector< float > & predicted,
    const std::vector< float > & actual)
```

Binary Cross Entropy Loss: $L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$.

Parameters

<i>predicted</i>	The output of the sigmoid layer (must be in range 0-1).
<i>actual</i>	The ground truth labels (0 or 1).

Returns

A vector of cross-entropy losses.

4.7.2.2 cross_entropy_derivative()

```
std::vector< float > cross_entropy_derivative (
    const std::vector< float > & predicted,
    const std::vector< float > & actual)
```

Derivative of Binary Cross Entropy Loss with respect to predictions.

Parameters

<i>predicted</i>	The output of the sigmoid layer (must be in range 0-1).
<i>actual</i>	The ground truth labels (0 or 1).

Returns

Gradient vector for backpropagation.

4.7.2.3 mean_squared_error()

```
std::vector< float > mean_squared_error (
    const std::vector< float > & predicted,
    const std::vector< float > & actual)
```

Computes the element-wise Squared Error loss: $L = \frac{1}{2}(pred - actual)^2$.

Parameters

<i>predicted</i>	The output predicted by the network.
<i>actual</i>	The ground truth target values.

Returns

A vector of squared differences.

4.7.2.4 mean_squared_error_derivative()

```
std::vector< float > mean_squared_error_derivative (
    const std::vector< float > & predicted,
    const std::vector< float > & actual)
```

Computes the derivative of the MSE loss with respect to the prediction.

$$\frac{\partial L}{\partial pred} = pred - actual$$

Parameters

<i>predicted</i>	The predicted output vector.
<i>actual</i>	The actual target vector.

Returns

Gradient vector for backpropagation.

4.7.2.5 mtanh()

```
std::vector< float > mtanh (
    const std::vector< float > & x)
```

Applies the Hyperbolic Tangent (Tanh) activation: $f(x) = \tanh(x)$.

Parameters

<i>x</i>	Input vector.
----------	---------------

Returns

Vector with tanh values in range (-1, 1).

4.7.2.6 relu()

```
std::vector< float > relu (
    const std::vector< float > & x)
```

Applies the Rectified Linear Unit (ReLU) activation: $f(x) = \max(0, x)$.

Parameters

<i>x</i>	Input vector of weighted sums.
----------	--------------------------------

Returns

Vector with ReLU applied element-wise.

4.7.2.7 relu_derivative()

```
std::vector< float > relu_derivative (  
    const std::vector< float > & x)
```

Derivative of the ReLU function: $f'(x) = 1$ if $x > 0$ else 0.

Parameters

x	Input vector.
---	---------------

Returns

Gradient vector.

4.7.2.8 sigmoid()

```
std::vector< float > sigmoid (  
    const std::vector< float > & x)
```

Applies the Sigmoid activation: $f(x) = \frac{1}{1+e^{-x}}$.

Parameters

x	Input vector.
---	---------------

Returns

Vector with sigmoid values in range (0, 1).

4.7.2.9 sigmoid_derivative()

```
std::vector< float > sigmoid_derivative (  
    const std::vector< float > & x)
```

Derivative of the Sigmoid function: $f'(x) = f(x) \cdot (1 - f(x))$.

Parameters

x	Input vector.
---	---------------

Returns

Gradient vector.

4.7.3 Variable Documentation

4.7.3.1 activation_map

```
const std::map<std::string, ActivationFunction> activation_map [extern]
```

Registry mapping string names (e.g., "relu") to [ActivationFunction](#) objects.

Registry mapping string names (e.g., "relu") to [ActivationFunction](#) objects.

Maps a string name (e.g., "sigmoid") to an [ActivationFunction](#) struct containing the primary function and its derivative.

4.7.3.2 loss_function_map

```
const std::map<std::string, LossFunction> loss_function_map [extern]
```

Registry mapping string names (e.g., "MSE") to [LossFunction](#) objects.

Registry mapping string names (e.g., "MSE") to [LossFunction](#) objects.

Maps a string name ("MSE" or "BCE") to a [LossFunction](#) struct containing the primary function and its derivative.

4.8 math_functions.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00005
00006 #include <vector>
00007 #include <string>
00008 #include <map>
00009 #include <functional>
00010
00016 std::vector<float> relu(const std::vector<float>& x);
00022 std::vector<float> relu_derivative(const std::vector<float>& x);
00023
00029 std::vector<float> sigmoid(const std::vector<float>& x);
00035 std::vector<float> sigmoid_derivative(const std::vector<float>& x);
00036
00042 std::vector<float> mtanh(const std::vector<float>& x);
00046 std::vector<float> mtanh_derivative(const std::vector<float>& x);
00047
00048
00055 std::vector<float> mean_squared_error(const std::vector<float>& predicted, const std::vector<float>&
actual);
00063 std::vector<float> mean_squared_error_derivative(const std::vector<float>& predicted, const
std::vector<float>& actual);
00064
00071 std::vector<float> cross_entropy(const std::vector<float>& predicted, const std::vector<float>&
actual);
00078 std::vector<float> cross_entropy_derivative(const std::vector<float>& predicted, const
std::vector<float>& actual);
00083 struct ActivationFunction {
00084     std::function<std::vector<float>(const std::vector<float>&)> func;
00085     std::function<std::vector<float>(const std::vector<float>&)> derivative;
00086 };
00091 struct LossFunction {
00092     std::function<std::vector<float>(const std::vector<float>&, const std::vector<float>&)> func;
00093     std::function<std::vector<float>(const std::vector<float>&, const std::vector<float>&)>
derivative;
00094 };
00096 extern const std::map<std::string, ActivationFunction> activation_map;
00098 extern const std::map<std::string, LossFunction> loss_function_map;
```

4.9 NeuralNetworkProjectFoCP/neural_network.h File Reference

Core Neural Network class definition and lifecycle management.

```
#include <vector>
#include <string>
#include <random>
```

Classes

- class [NeuralNetwork](#)
Implementation of a multilayer perceptron with backpropagation capabilities.

4.9.1 Detailed Description

Core Neural Network class definition and lifecycle management.

4.10 neural_network.h

[Go to the documentation of this file.](#)

```
00001 #ifndef NEURAL_NETWORK_H
00002 #define NEURAL_NETWORK_H
00006 #include <vector>
00007 #include <string>
00008 #include <random>
00009
00014 class NeuralNetwork {
00015 public:
00016     NeuralNetwork();
00028     void initialize_weights_and_biases(int input_size, int hidden_layers_number, int
neurons_per_hidden_layer, int output_size, std::string initialization_method = "Xavier",
00029         std::vector<std::string> activation_functions = {},
00030         int seed = std::random_device()());
00034     void visualize_model() const;
00041     std::vector<std::vector<float>> feedforward(const std::vector<std::vector<float>>& input) const;
00052     void train(const std::vector<std::vector<float>>& inputs,
00053         const std::vector<std::vector<float>>& targets,
00054         const std::vector<std::vector<float>>& test_inputs,
00055         const std::vector<std::vector<float>>& test_targets,
00056         int epochs,
00057         float learning_rate,
00058         const std::string& loss_function_name);
00066     float test_model(const std::vector<std::vector<float>>& test_inputs,
00067         const std::vector<std::vector<float>>& test_targets,
00068         const std::string& loss_function_name) const;
00073     void save_model_to_file(const std::string& filename) const;
00079     void load_model_from_file(const std::string& filename);
00080
00081     // Getters (mainly for testing)
00082     const std::vector<std::vector<std::vector<float>>>& get_weights() const;
00083     const std::vector<std::vector<float>>& get_biases() const;
00084     const std::vector<std::string>& get_activations() const;
00085
00086 private:
00087     std::vector<std::vector<std::vector<float>>> weights;
00088     std::vector<std::vector<float>> biases;
00089     std::vector<std::string> activations;
00090
00091     bool is_model_valid = false;
00092     void validate_model_structure();
00093
00094 };
00095 #endif
```


4.11 resource.h

```
00001 //{NO_DEPENDENCIES}
00002 // used by: NeuralNetworkProjectFoCP.rc
00003 //
00004 #define IDR_MENU1 101
00005 #define ID_HEJ_HWEH 40001
00006
00007 // Next default values for new objects
00008 //
00009 #ifdef APSTUDIO_INVOKED
00010 #ifndef APSTUDIO_READONLY_SYMBOLS
00011 #define _APS_NEXT_RESOURCE_VALUE 102
00012 #define _APS_NEXT_COMMAND_VALUE 40002
00013 #define _APS_NEXT_CONTROL_VALUE 1001
00014 #define _APS_NEXT_SYMED_VALUE 101
00015 #endif
00016 #endif
```

4.12 resource.h

```
00001 //{NO_DEPENDENCIES}
00002 // Microsoft Visual C++ generated include file.
00003 // Used by UnitTest1.rc
00004
00005 // Następane wartości domyślne dla nowych obiektów
00006 //
00007 #ifdef APSTUDIO_INVOKED
00008 #ifndef APSTUDIO_READONLY_SYMBOLS
00009 #define _APS_NEXT_RESOURCE_VALUE 101
00010 #define _APS_NEXT_COMMAND_VALUE 40001
00011 #define _APS_NEXT_CONTROL_VALUE 1001
00012 #define _APS_NEXT_SYMED_VALUE 101
00013 #endif
00014 #endif
```

4.13 pch.h

```
00001 // pch.h: wstępnie skompilowany plik nagłówka.
00002 // Wymienione poniżej pliki są kompilowane tylko raz, co poprawia wydajność kompilacji dla przyszłych
00003 // kompilacji.
00004 // Ma to także wpływ na wydajność funkcji IntelliSense, w tym uzupełnianie kodu i wiele funkcji
00005 // przeglądania kodu.
00006 // Jednak WSZYSTKIE wymienione tutaj pliki będą ponownie kompilowane, jeśli którykolwiek z nich
00007 // zostanie zaktualizowany między kompilacjami.
00008 // Nie dodawaj tutaj plików, które będziesz często aktualizować (obniża to korzystny wpływ na
00009 // wydajność).
00010
00011 #ifndef PCH_H
00012 #define PCH_H
00013
00014 // w tym miejscu dodaj nagłówki, które mają być wstępnie kompilowane
00015
00016 #endif //PCH_H
```

